

時間軸への対応 — フレームを進めて記録する

2026-09-13 | Houdini 21.0.700 | /obj/time_axis | ノード 4 / 接続 2

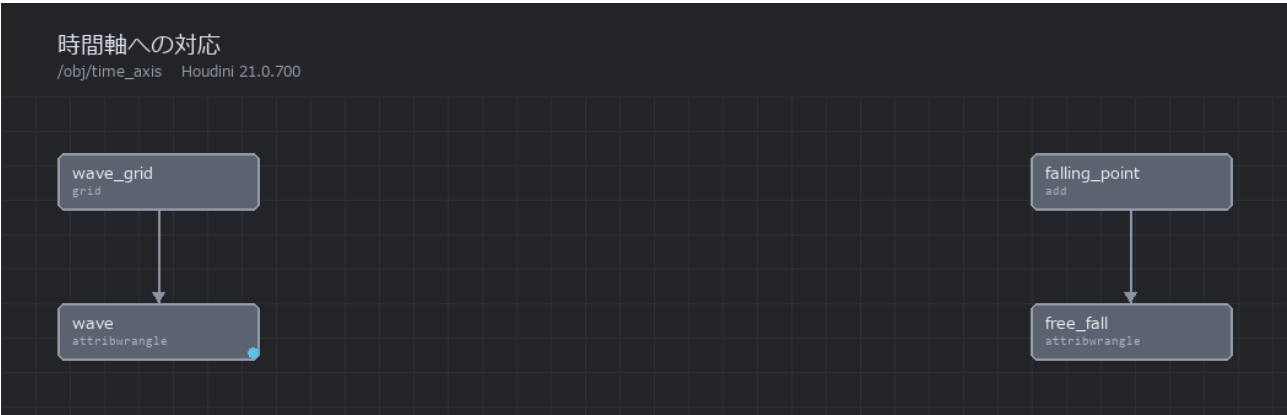
何を作ったか

ここまでの12実験はすべて1フレームだけを見ていた。エフェクトは時間変化そのものが結果なので、フレームを進めて連番で書き出し、1枚のコンタクトシートと動くGIFにまとめる仕組みが要る。

仕組みが正しいかは、答えが分かっている動き（自由落下の式）で確かめた。

あわせて、この先のシミュレーションで使うノードがどこにあるのかを数えた。その結果、当初の予定を変えるべきだと分かった。

ノードの接続

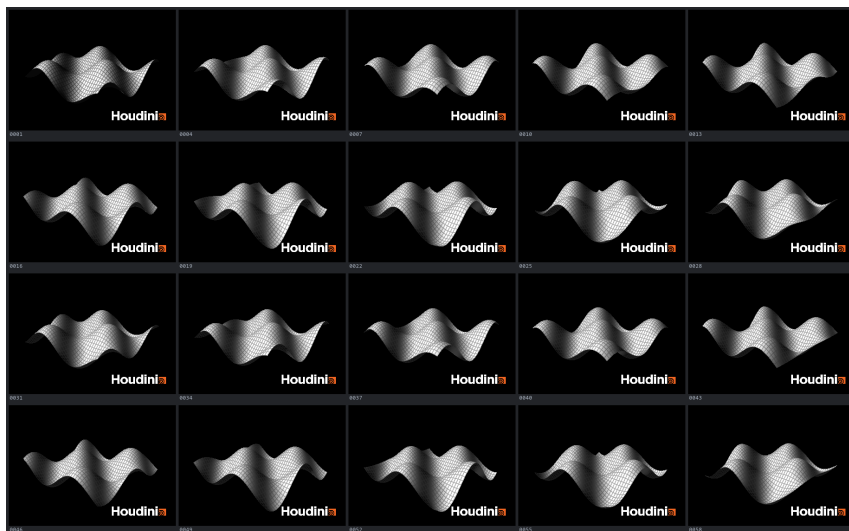


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
wave_grid	grid	cols = 40 rows = 40
wave	attribwrangle	snippet = @P.y = sin(@P.x * 0.8 + @Frame * 0.2) * cos(@P.z * 0.8) * 1.5;
falling_point	add	points = 1
free_fall	attribwrangle	snippet = float t = (@Frame - 1) / 24.0; @P.y = 20.0 - 0.5 * 9.8 * t * t;

連番の書き出し（20コマ）

フレーム1から58まで3コマおき。カメラは全フレームを通して固定してある。動いているのか大きさが変わっているのかを画から判断できるようにするため。



20コマを1枚に並べたコンタクトシート。左下の数字がフレーム番号

検証: 自由落下の式と一致するか

高さ20から重力9.8で落ちる点を作り、理論値 $h = \frac{1}{2}gt^2$ と比べた。24コマで1秒として計算している。

フレーム	経過秒	測った高さ	理論値	差
1	0.0000	20.00000	20.00000	0.0000000
10	0.3750	19.31094	19.31094	0.0000004
20	0.7917	16.92899	16.92899	0.0000002
30	1.2083	12.84566	12.84566	-0.0000005
40	1.6250	7.06094	7.06094	-0.0000006

シミュレーション関連ノードがどこにあるか

SOP は形を作る階層、DOP はシミュレーションの階層。どちらに何種類あるかを数えると、どこから手を付けるべきかが見えてくる。

対象	キーワード	SOP にある数	DOP にある数
パーティクル	pop	0 種	67 種
剛体（破壊）	rbd	47 種	21 種
布	vellum	22 種	7 種
煙・炎	pyro	14 種	3 種
液体	flip	8 種	4 種

わかったこと

- ・連番の書き出しは正しく動いている。自由落下の式との差は最大でも 0.0000006 で、これは小数の計算精度そのもの。フレームを進めて記録する仕組みに間違いはない。
- ・20枚の書き出しに 1.18 秒（1枚あたり 59 ミリ秒）。60フレーム全部でも3.5秒程度で、実用的な速さ。レンダリングの重さは問題にならない。
- ・カメラは全フレームを覆う範囲で一度だけ決める。フレームごとに枠を合わせ直すと、対象が動いているのか大きさが変わっているのかを画から判断できなくなる。そのために、全フレームを走査して範囲を

求める関数を用意した。

- ・出力は2つの形にした。PDFには動きを載せられないのでコンタクトシート（全コマを1枚に並べてフレーム番号を振る）、HTMLにはGIF。同じ連番から両方を作る。

- ・パーティクルは SOP に1つもない（0種）。すべて DOP

側にある（67種）。つまりパーティクルを扱うには DOP

ネットワークを組む必要があり、他の実験のようにSOPをつなぐだけでは始められない。

- ・一方で剛体（RBD）は SOP に 47種もある。布（Vellum）も

22種。SOPレベルのラッパーが充実している分野の方が、入口としては易しい。

当初はパーティクルから始める予定だったが、RBD か Vellum から入る方が理にかなっている。

- ・煙・炎（Pyro）は SOP 14種 / DOP 3種、液体（FLIP）は SOP 8種 / DOP 4種。どちらも SOP 側にラッパーはあるが数は少ない。

- ・ solver SOP

を使って自分で重力を書こうとしたが、これはロックされたアセットで、中にノードを作るには `allowEditingOfContents()` が必要。しかも中身は `dopnet` と4つの `null` で、想像していた「前フレームの結果を受け取って加工する」という単純な構造ではなかった。プログラムから組むには追加の調査が必要

。

- ・そのため今回は、自由落下を数式で書いて仕組みの検証にとどめた。これはシミュレーションではない（前のフレームの結果を使っていない）。本物のシミュレーションは次の実験から。

次にやること

- ・ RBD（剛体）から始める。SOPレベルのノードが47種あり、boolean の Shatter で作った破片をそのまま落とせる

- ・ Vellum（布）も SOPレベルのノードが揃っているので入りやすい

- ・シミュレーションの結果をファイルに保存して使い回す（キャッシュ）。毎回計算し直すと実験が遅くなる